

vToken: Token-Level Virtualization for Reclaimable KV Caches

Yuanhang Gao
National University of Defense
Technology
Changsha, China
gaoyh@nudt.edu.cn

Xiangrui Yang
National University of Defense
Technology
Changsha, China
yangxiangrui11@nudt.edu.cn

Yuanfeng Chen
National University of Defense
Technology
Changsha, China
chenyuanfeng@nudt.edu.cn

Hongjia Chen
National University of Defense
Technology
Changsha, China
chenhongjia22@nudt.edu.cn

Qianru Lv
National University of Defense
Technology
Changsha, China
lvqianru11@nudt.edu.cn

Wenfei Wu
Peking University
Beijing, China
wenfeiwu@pku.edu.cn

Dongsheng Li*
National University of Defense
Technology
Changsha, China
dsli@nudt.edu.cn

Abstract

Large language model serving faces a critical memory bottleneck: the KV cache grows with sequence length and batch size. PagedAttention uses fixed-size memory blocks to reduce allocator-level fragmentation, but recent KV eviction algorithms operate at a token granularity finer than block-level management. This mismatch causes intra-block fragmentation, leaving a large fraction of allocated KV memory unreclaimable. We present vToken, a lightweight token-level virtualization layer that decouples logical token liveness from physical block placement. vToken maintains a stable logical token view through token-table indirection and realizes physical reclamation by repacking live tokens asynchronously. The design preserves PagedAttention kernels and CUDA Graph compatibility. We implement vToken in vLLM and evaluate it with H2O, Random, and Scissorhands across models. Compared with a paired NAIVE-EVICT baseline, vToken reduces retained KV blocks per request by 27.2%–72.3% and improves SLA-constrained throughput by up to 1.37 $\times$. Under a constrained active-KV budget, it extends the maximum feasible concurrency by up to 2 $\times$, while reducing the per-policy integration footprint from 500+ lines to under 50.

Keywords: Large language models, KV Cache, Intra-block fragmentation, memory management

1 Introduction

The rapid advancement of large language models (LLMs) [1, 12, 19] has created an urgent demand for efficient inference serving systems. From conversational chatbots [7, 8]

to code generation [4, 13, 26], LLMs are now deeply integrated into a wide spectrum of applications [27]. However, high-throughput, low-latency inference faces a critical bottleneck: the KV cache. During autoregressive generation, the KV cache stores the key and value tensors computed at each step for reuse in subsequent token generation, reducing attention complexity from $O(N^2)$ to $O(N)$. The size of the KV cache scales linearly with both context length and batch size. For long-context scenarios, KV cache can occupy tens of gigabytes of GPU memory, far exceeding the memory footprint of model weights. Consequently, KV cache management directly dictates the achievable concurrency and throughput of LLM serving systems [3, 17, 30].

PagedAttention [17] and its implementation in vLLM have improved KV cache management by partitioning memory into fixed-size blocks, drawing inspiration from virtual memory paging [16] in operating systems. This block-based design reduces allocator-level fragmentation and enables cache sharing across requests. PagedAttention therefore serves as the *block-level substrate* in our design: it provides efficient physical KV allocation and attention-kernel compatibility, while exposing only a block-granular interface for allocation and reclamation.

Token-level KV eviction algorithms reduce memory pressure by exploiting the observation that not all historical tokens are equally important for future generation [33]. A diverse family of strategies has emerged: H2O [36] retains “heavy hitter” tokens with the highest cumulative attention scores; StreamingLLM [33] preserves attention sink tokens plus a recent window; Scissorhands [22] and FastGen [11] propose further criteria based on sparse attention patterns.

*Corresponding author.

However, although these techniques are effective individually, their naive composition with block-managed runtimes exposes a granularity mismatch: eviction policies decide which tokens should be retained or evicted, while the serving runtime maps, remaps, and reclaims KV memory at block granularity. This leaves a missing layer between token-level eviction decisions and block-level KV management.

This mismatch is not merely a matter of wasted space, but a missing runtime boundary between token-level liveness and block-level reclamation. When a block contains both evicted and retained tokens, the block cannot be released, so its internal holes cannot be reused by other requests. Our preliminary evaluation shows that running token-level eviction on vLLM at a 16K context leaves most allocated blocks at no more than 50% utilization across policies and workloads, with resulting intra-block waste exceeding 40%. As a result, a substantial portion of GPU memory remains trapped in allocated but underutilized blocks, limiting effective serving capacity under memory pressure. Beyond fragmentation, this token/block granularity mismatch creates a barrier to algorithm integration. Integrating a new token-level eviction algorithm requires developers to understand block-manager internals and manually translate token-level retention semantics into block-level actions. This typically leads to two suboptimal outcomes: either waiting for an entire block to become empty, which sacrifices algorithm effectiveness, or modifying memory management behavior, which tightly couples policy logic with runtime code. This coupling slows algorithm iteration and makes it difficult to accommodate diverse future eviction strategies.

A straightforward workaround is to reduce block size, for instance from 16 to 8 or 4 tokens. However, smaller blocks increase metadata overhead, address-mapping complexity, and the number of fine-grained non-contiguous KV transfers, which can reduce bandwidth efficiency in attention, copy, or offload paths [34]. At the other extreme, pure token-level memory management—allocating space independently for each token—is impractical due to GPU memory allocation granularity and prohibitive metadata overhead. Thus, a new abstraction is needed that retains the efficiency of block-based management while supporting token-level semantics.

Our core idea is to introduce vTOKEN, a lightweight token-level memory virtualization layer between eviction policies and the PagedAttention substrate. This layer defines a clear semantic boundary: policies decide which tokens to evict, while the runtime maintains a token-level logical address space over block-managed KV memory. Upward, vTOKEN exposes a token-level interface that lets policies operate without reasoning about physical blocks. Downward, it maintains logical-to-physical mappings, remaps attention slots, and reclaims physical blocks by asynchronously repacking live tokens when fragmentation becomes actionable. vTOKEN’s primary contribution is the token-level virtualization abstraction; its physical reclamation backend realizes this boundary,

using lazy compaction as the relocation mechanism in our implementation. vTOKEN is a pressure-activated extension rather than a replacement for native full-KV serving. The native path remains preferable when KV memory is not the bottleneck; under memory pressure, vTOKEN provides the runtime boundary that turns token-level KV eviction into reusable physical KV capacity.

This paper makes the following contributions:

- We identify the mismatch between token-level KV eviction and block-level KV cache management as a missing runtime abstraction layer and quantify its memory cost (§2.3).
- We propose vTOKEN, a token-level virtualization layer that decouples logical token liveness from physical block placement. vTOKEN exposes a stable per-sequence token view upward and provides physical reclamation downward, without attention-kernel modifications.
- We implement vTOKEN in vLLM and evaluate it across models and three eviction policies. vTOKEN reduces retained KV blocks by 16%–85% and improves SLA-constrained throughput by up to 1.37× over NAIVE-EVICT; under a constrained active-KV budget, it extends the maximum feasible concurrency by up to 2× and reduces per-policy integration from 500+ to under 50 lines of code.

2 Background and Motivation

In this section, we provide the necessary background on KV cache management in LLM serving and token-level eviction algorithms. We then identify and quantify the granularity mismatch that motivates our work.

2.1 KV Cache and PagedAttention

Modern autoregressive LLMs generate tokens sequentially. At each decoding step, the model computes attention over all previous tokens, requiring the Key and Value tensors of those tokens to be stored in GPU memory, i.e., the KV cache. For a model with L layers, H attention heads, and head dimension D , the KV cache size per token is $2 \times L \times H \times D \times \text{dtype_size}$. For a 13B-parameter LLaMA model with 40 layers and 5120 hidden dimension, each token consumes approximately $2 \times 40 \times 5120 \times 2 \text{ B} \approx 0.8 \text{ MB}$. With a 32K-token context, the KV cache alone exceeds 25 GB, dominating the memory footprint of inference.

To manage this large cache efficiently, vLLM [17] introduced PAGEDATTENTION, which organizes the KV cache into fixed-size *blocks* (typically 16 tokens per block). PagedAttention maintains a block table that maps logical blocks to physical GPU memory blocks, analogous to virtual memory paging in operating systems. By allocating blocks on demand and allowing non-contiguous physical storage, it reduces external fragmentation; shared-prefix reuse further reduces memory consumption. As a result, PagedAttention

has been widely adopted and serves as the foundation for many LLM inference engines [24, 25].

2.2 KV Eviction Algorithms

Recent works have proposed diverse token-level eviction strategies that exploit attention sparsity, each with different criteria for determining token importance: H2O [36] (Heavy-Hitter Oracle) tracks the cumulative attention score of each token and retains only those with the highest scores—the “heavy hitters,” reporting up to 50% cache reduction with limited perplexity increase. Sliding window policies retain only the most recent N tokens, discarding all earlier history. StreamingLLM [33] preserves attention sink tokens (typically the first few tokens that accumulate high attention mass) plus a recent window. Random eviction probabilistically discards tokens to meet memory budgets. Scissorhands [22] and FastGen [11] propose more sophisticated criteria based on attention patterns and token roles. Despite their algorithmic diversity, these approaches share a common requirement: they operate at the granularity of *individual tokens*, making per-token retention decisions. When integrated into a serving system, they can reduce KV memory consumption and thereby support higher throughput or longer context windows.

2.3 The Granularity Mismatch Problem

Despite the benefits of both PagedAttention and token-level eviction, a fundamental mismatch exists between block-level memory management and token-level KV eviction. We first formalize this mismatch, then quantify its impact through preliminary experiments, and discuss its implications for algorithm integration.

2.3.1 Formal Definition. Let each block have capacity S . For a physical block b , let n_b denote the number of valid (non-evicted) tokens stored in that block, and define block utilization as $u_b = n_b/S$. Given N allocated blocks, the intra-block waste ratio is

$$F = 1 - \frac{\sum_{i=1}^N n_i}{N \cdot S} = 1 - \frac{1}{N} \sum_{i=1}^N u_i.$$

When token-level eviction is applied, some tokens in a block become invalid, reducing n_b while the block itself remains allocated. This creates internal fragmentation: memory is reserved but underutilized.

2.3.2 Quantifying the Impact. To understand the severity of this fragmentation, we conduct a preliminary experiment using vLLM integrated with a H2O-inspired token-level eviction algorithm (referred to as H2O hereafter). We run the Llama-3.1-8B model on a 16K-token subset of the ShareGPT [10] and LongBench [5] datasets with a batch size of 16. We measure the fraction of allocated blocks that remain low-utilization after token-level eviction.

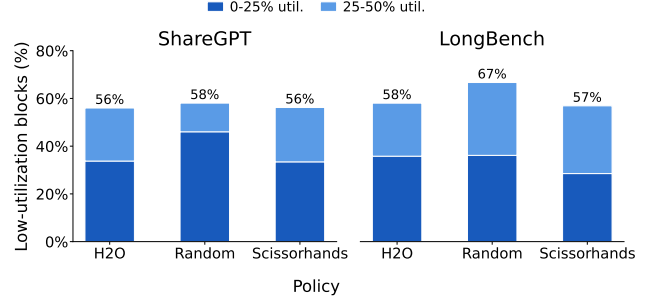


Figure 1. Low-utilization KV blocks under token-level eviction. Bars show allocated blocks with $\leq 50\%$ utilization, split into 0–25% and 25–50% bins; such partially live blocks remain unreclaimable in a block-granular runtime.

Results. Figure 1 shows that token-level eviction leaves a large fraction of allocated blocks at low utilization. Across policies and workloads, most allocated blocks are at no more than 50% utilization, and the resulting intra-block waste ratio F reaches 40–60%. These blocks cannot be returned to the free pool because each still contains at least one retained token. Thus, token-level eviction reduces logical KV demand, but without a token-level virtualization layer, much of this reduction remains trapped as partially live physical blocks rather than reusable capacity.

2.3.3 Algorithm Integration Barrier. Beyond memory waste, the granularity mismatch imposes a high cost on integrating new eviction algorithms into existing systems. Without a virtualization layer, a vLLM integration must compute token importance in the scheduler, select tokens to evict, translate those token-level decisions into block-level actions, and keep attention slot mappings consistent with any retained tokens. Because blocks cannot be partially freed, the implementer must either wait until an entire block becomes empty, delaying reclamation, or add custom logic to evacuate live tokens before releasing a block, which requires deep changes to the block allocator and careful synchronization.

This process requires repeated engineering effort for each new algorithm and complicates production integration. In our experience, a direct integration of a single token-level eviction algorithm (e.g., H2O) into vLLM without a virtualization layer requires modifying over 500 lines of code across multiple core modules, with additional debugging effort to ensure correctness; each additional strategy requires comparable work.

3 vToken Design

3.1 Design Goals and Virtualization Boundary

The granularity mismatch in §2 stems from a single missing piece: the runtime does not expose token-level liveness as a first-class concept. Two direct fixes are insufficient. A fully token-granular allocator would expose the right semantic unit, but it weakens the contiguous-access assumptions of

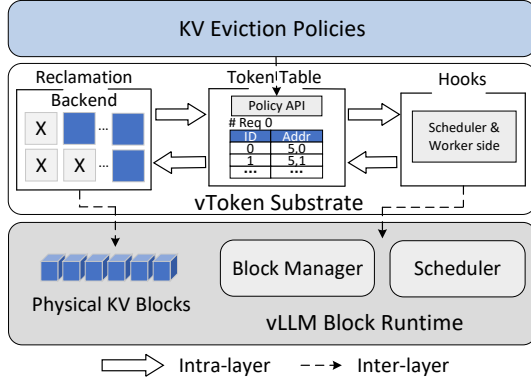


Figure 2. Overview of the **vToken** system

PagedAttention kernels, multiplies per-entry metadata, and would require invasive CUDA changes across systems such as vLLM, TensorRT-LLM, and SGLang [37]. Shrinking the block size redistributes the same fragmentation–bandwidth tradeoff without removing it. vTOKEN therefore keeps the block-managed KV runtime intact and adds a thin *virtualization boundary* that exposes token-granular semantics on top of it, leaving allocator and attention kernels untouched.

The boundary defines a contract between two views of the same request. Above it, eviction policies operate on logical token identities: they emit *which* tokens are no longer needed and never reason about physical blocks, attention slots, or reclamation timing. Below it, the runtime maintains a token-level logical address space, owns the logical-to-physical mapping, refreshes the slot mappings that attention kernels consume, and decides *when* reclaiming fragmented blocks is profitable. Crucially, this contract decouples *deciding* a token is dead from *reclaiming* its KV memory, which lets reclamation be deferred and batched in the background.

Maintaining this contract under live decoding raises three challenges that any realization of the boundary must solve. (C1) *Dual-view consistency*. A token may become dead independently of its neighbors, but a block can be released only when all useful KV entries inside it have been relocated or discarded; the runtime needs explicit metadata that simultaneously captures token liveness and block occupancy. (C2) *Safe reclamation during decoding*. Relocating KV entries changes the physical slots that attention reads, so slot mappings must be refreshed and asynchronous copies synchronized before any dependent attention kernel runs—without adding a global synchronization point to every decoding step. (C3) *Policy-neutral, amortized cost*. Different policies (H2O, StreamingLLM, Random) must reuse the same scheduler, block-manager, and worker-layout code, and indirection or reorganization cost must be paid only when fragmentation is actionable.

Figure 2 shows how vTOKEN answers each challenge: a per-sequence *token table* provides the dual-view metadata for C1

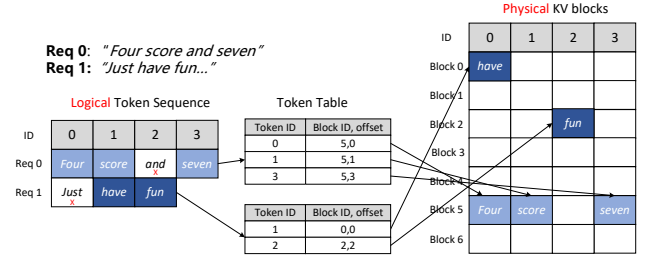


Figure 3. Token table as virtualization metadata for the logical token address space

(§3.2); a *physical reclamation backend* batches reclamation in the background to satisfy C3 and to drive the asynchronous copies of C2 (§3.3); and *reclamation-aware scheduler hooks* insert the slot-mapping refresh and CUDA-event dependency that close C2’s safety requirement (§3.4).

3.2 Token Table and Logical Address Space

The **token table** is the central metadata structure of vTOKEN (Figure 3). It is maintained per request and records, for each logical token ID, the current physical location as a tuple (block ID, offset) along with a liveness bit. Marking a token reclaimable updates only this metadata; no KV memory is moved or freed until the reclamation backend acts. This indirection separates policy semantics (what to evict) from physical layout maintenance (how to repack live tokens). The table exposes two interfaces to upper-layer eviction policies, plus one internal interface used by the reclamation engine:

- `evict_token(req_id, token_id)`: Marks the token as logically inactive. The KV entry may remain physically present until reclamation, but it is excluded from subsequent attention slot mappings and treated as dead space by the reclamation planner.
- `sync_new_tokens(req_id, block_ids, total_len)`: Registers newly generated tokens. The scheduler passes the request’s current block list and updated sequence length; the table assigns sequential logical IDs and records their physical positions.
- `apply_moves(req_id, moves)`: Invoked by the reclamation backend after copies complete. Each move carries a token ID and its new (block, offset); the table atomically updates the affected entries so subsequent slot lookups see the post-relocation layout.

These interfaces are deliberately block-agnostic: a policy calls `evict_token` without knowing which block holds the token or whether the block can be freed. This makes vTOKEN a reusable substrate for diverse token-level policies. The metadata cost is modest— $O(L)$ per sequence of length L (one mapping entry plus liveness state per token), independent of the number of layers, heads, or KV tensor elements; §4 reports the absolute footprint.

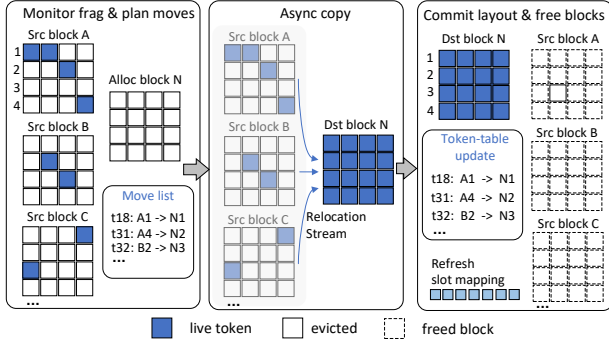


Figure 4. Physical reclamation workflow

The canonical table lives on the CPU for simplicity and compatibility, while a GPU-resident lookup cache accelerates slot translation in the steady-state decoding path. After eviction or relocation, the cached translation array is refreshed before constructing the next attention slot mapping. When a sequence grows append-only, the cache is updated by appending the new entries; a full rebuild is triggered only on structural changes (eviction, relocation, or layout reconciliation).

3.3 Physical Reclamation: A Backend for the Virtualization Boundary

The reclamation backend realizes the virtualization boundary by translating logical token liveness into safe physical block reuse. In our implementation, lazy compaction is the concrete relocation mechanism: it repacks retained tokens from low-utilization blocks, updates the token table after copies complete, and returns emptied blocks to the allocator. Throughout relocation, the token table preserves a stable logical token view while physical KV locations change underneath. The backend consists of four stages—reclamation eligibility, headroom-aware admission, relocation planning, and asynchronous copy—as shown in Figure 4.

3.3.1 Reclamation Eligibility. The backend uses per-block liveness metadata to decide when logical holes have become physically actionable, without exposing block state to the eviction policy. For each block b , it maintains n_b , the number of live tokens in that block (i.e., tokens not marked as evicted via `evict_token`), and derives the intra-block waste ratio F defined in §2.3.1. A high F indicates that many allocated blocks are underutilized. The metadata is updated incrementally on token eviction and block allocation/free events, so eligibility checks do not scan the full token table. To avoid excessive overhead, we amortize checks over multiple scheduling iterations and rate-limit reclamation attempts, so the planner is invoked only when fragmentation is likely to be actionable.

3.3.2 Headroom-Aware Admission. Out-of-place relocation needs destination blocks before fragmented source blocks can be released. Waiting until the free list is empty would therefore prevent reclamation from starting precisely when memory pressure is highest. vToken attempts reclamation only when the global intra-block waste ratio F exceeds a threshold θ_F , enough low-utilization blocks exist, and the free-block count is near a low watermark. The threshold θ_F is tunable. A lower threshold attempts reclamation more frequently, potentially keeping fragmentation lower but incurring more overhead; a higher threshold reduces attempt frequency at the cost of higher average fragmentation. Through empirical exploration (§5.6), we set $\theta_F = 0.25$ as a default, which balances memory efficiency and overhead in our experiments. The scheduler maintains bounded evacuation headroom inside the same KV block budget used for admission. This headroom is temporary working space for destination blocks, not an additional memory pool. When the free-block count approaches the low watermark, the scheduler prioritizes reclamation and applies admission backpressure instead of consuming the remaining free blocks with new requests. A reclamation plan is admitted only if the reserved headroom can provide the required destination blocks; otherwise, the plan is deferred until blocks become available.

3.3.3 Relocation Planning. Planning remains request-local. For each request, the backend selects eligible low-utilization source blocks, computes the live-token footprint, and constructs a relocation plan only when the projected block reduction is positive. The batch size B remains a configurable upper bound on the number of source blocks considered in one planning round. Let the selected source blocks contain a total of T_{live} live tokens.

The backend then allocates destination blocks from the bounded evacuation headroom and the remaining free block pool. The number of destination blocks needed is $\lceil T_{\text{live}}/S \rceil$. A plan is admitted only if these destinations are available and the source batch can release more blocks than it consumes as destinations; otherwise, the planner reduces the candidate set and emits no relocation plan for that round. This admission rule keeps reclamation profitable while preventing relocation from borrowing unbounded memory under pressure. The resulting move list iterates through the live tokens in the source blocks (preserving logical token order when possible to maintain cache locality) and assigns them sequentially to the destination blocks, filling each destination block completely before moving to the next. Each move records a token ID, its source location (`src_block`, `src_offset`), and its destination location (`dst_block`, `dst_offset`); applying the list updates the token table after copies complete.

3.3.4 Stage-Aware Asynchronous Copy. A decode step decomposes into three GPU phases with distinct resource profiles: (a) *attention forward*, which reads the KV cache and

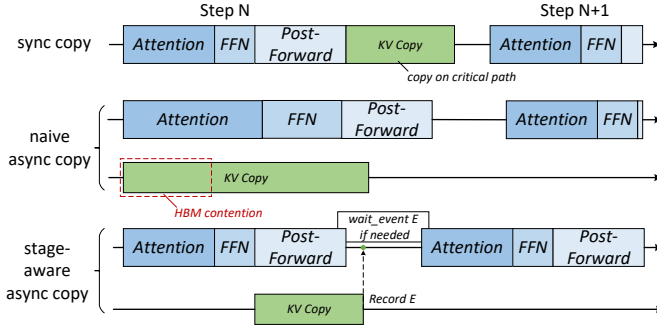


Figure 5. Stage-aware asynchronous copy. vTOKEN avoids putting KV copy on the critical path and reduces HBM contention by launching copy after forward; a CUDA event guards the next attention step only when needed.

is HBM-bandwidth bound; (b) *FFN forward*, which is compute bound; and (c) *post-forward work* (sampling, scheduling, next-step preprocessing), which is dominated by CPU-side logic with low GPU utilization. Naïvely overlapping relocation copy with the entire step causes contention in phase (a): both attention and KV-copy compete for HBM bandwidth, and the attention kernel slows down whenever a copy is in flight.

As shown in Figure 5, vTOKEN avoids this by launching KV copies *after* the current step’s forward returns, on a dedicated relocation stream. The current step’s slot mappings have already been built from the old token table and consumed by the in-flight forward, so the post-forward token-table update cannot corrupt that step. The copies then proceed concurrently with sampling, scheduling, and next-step preprocessing—phases that do not read the KV cache and therefore do not contend with the relocation stream for HBM bandwidth. The next step’s forward inserts a stream-level CUDA event dependency before model invocation; if the copy completes during post-forward work, the fence is a no-op. Because the dependency is expressed as a GPU-side `wait_event` rather than a host synchronization, the CPU never stalls in the async path.

Source blocks are returned to the free pool only after the new packed layout has been committed, and logical layout metadata is updated in the worker path after forward returns. This stage-aware schedule preserves the existing execution structure of vLLM while ensuring that subsequent attention steps observe a consistent post-relocation KV view without sharing HBM bandwidth with the attention kernel.

3.4 Integration with the Scheduler

Integrating vTOKEN with vLLM’s scheduler requires modifications in two key areas: slot mapping and scheduling hooks.

Slot mapping. In vLLM, attention kernels access the KV cache via *slots*, which are linearized indices computed as

$slot = block_id \times block_size + offset$. With the introduction of the token table, this direct computation is no longer valid because tokens may have been moved during relocation. Instead, we modify the slot mapping function to consult the token table:

$$slot = token_table[token_id].block_id \times S + token_table[token_id].offset$$

This indirection makes attention slot construction follow the current physical location of each retained token after relocation. The lookup is performed once per token during input preparation rather than inside the attention kernel.

Scheduler hooks. We integrate reclamation into vLLM’s scheduling loop through a small set of scheduler- and worker-side hooks:

- **Scheduler-side reservation hook:** Maintains the bounded evacuation headroom described in §3.3 before admitting new KV allocations.
- **Worker-side reclamation hook:** During the worker execution path, the runtime evaluates reclamation opportunities, launches asynchronous KV-copy operations for selected requests, and rewrites the corresponding logical layout metadata.
- **Pre-attention synchronization hook:** Before attention consumes KV entries that may have been relocated, the worker inserts a stream-level dependency on the relocation event, ensuring that the moved KV data are visible to subsequent kernels.

Compatibility with CUDA Graph. vTOKEN does not disable CUDA Graphs or recapture graphs after relocation. The KV cache tensors and graph-captured input buffers remain stable; only the contents of the mutable slot-mapping buffer are updated before replay according to the token table. Relocation runs outside the captured graph on a separate CUDA stream, and the worker inserts a CUDA-event dependency before any replay that may read relocated KV entries. Thus, vTOKEN preserves CUDA Graph execution through dynamic slot-mapping adjustment rather than graph bypass or graph recapture.

3.5 Correctness Invariants

The components in §3.2–§3.4 are coordinated by four invariants that together define what “correctness under background relocation” means in vTOKEN. The mechanisms enforcing each invariant have been introduced above; here we state them explicitly so that the validation experiments in §5.8 can be read as direct tests.

- **I1 (Token conservation).** For each request, the set of active logical token IDs and their associated K/V tensor contents are preserved across every relocation event. Relocation may change the physical (block, offset) of a token but never its logical identity or its KV payload.

- **I2 (Unique mapping).** Every active logical token maps to exactly one (block, offset), and every occupied physical slot is referenced by at most one logical token. The token table enforces this on `apply_moves` by atomically clearing the source slot and writing the destination slot.
- **I3 (Pre-attention visibility).** A relocated KV entry is read by attention only after the corresponding copy has completed. For each committed relocation, the worker records a CUDA event on the copy stream and inserts a stream-level wait before any subsequent attention launch whose slot mapping may reference the relocated entries. This dependency replaces a global GPU synchronization.
- **I4 (Layout-aware planning).** A relocation plan is committed only from a consistent block-list and liveness snapshot of the affected requests, never overlaps an in-flight plan on the same request, and is rejected when the projected post-relocation block count does not strictly decrease. Shared-prefix blocks are excluded from plans and handled conservatively as discussed in §6.

Invariants I1 and I2 are checked online for every relocation event during evaluation (§5.8). I3 is structural: it follows from the CUDA-event topology established in §3.3–§3.4 and is asserted in unit tests rather than measured. I4 is enforced by the planner’s profitability and isolation gates described in §3.3.

4 Implementation

Our vToken prototype is implemented on top of vLLM (v0.18.0), using PyTorch v2.10.0. It adds three components: a per-sequence TokenTable for logical-to-physical mappings, a ReclamationManager for fragmentation monitoring and relocation planning, and a CUDACopyEngine for asynchronous KV movement on a separate CUDA stream. Token-table space is separate from these runtime costs. The table is maintained per sequence rather than per layer; even with a conservative 16-byte entry, a 16K-token sequence requires only 256 KB of metadata, below 0.1% of the roughly 2 GB FP16 KV cache footprint for the evaluated 7B/8B models.

The vLLM-based prototype requires scheduler and worker hooks for headroom reservation, slot mapping, relocation launch, and pre-attention synchronization. The worker maintains cached slot-translation arrays and refreshes them only after structural layout changes. KV relocation is launched on a non-blocking CUDA stream, while the main stream waits on CUDA events before any attention launch that may consume relocated entries. The prototype targets the single-node, single-GPU decoding fast path, isolating the interaction between token-level eviction and block-managed KV allocation without conflating it with distributed scheduling or cross-device KV movement. Per-block liveness metadata let the planner find underutilized blocks without scanning the full table; a greedy packer with conservative gating triggers relocation only when the projected gain exceeds a threshold.

With this runtime layer in place, policy integration becomes token-centric rather than block-centric. A policy only needs to report newly appended tokens and selected victims; for example, H2O calls `sync_new_tokens()` for generated tokens and `evict_token()` for tokens whose scores fall below its budget. vToken then updates the token table, refreshes slot mappings, and schedules physical reclamation without requiring the policy to manipulate vLLM blocks directly.

Integration complexity. Figure 6 summarizes the resulting adapter path and code footprint: policies define only `SELECTVICTIMS`, while token-table updates, slot remapping, and reclamation are shared runtime functions.

vToken policy-adapter procedure

1	<code>SYNCNEWTOKENS(r, B_{new}, L)</code>	shared adapter
2	<code>$V \leftarrow P.\text{SELECTVICTIMS}(r)$</code>	policy hook
3	for each $t \in V$: <code>EVICTTOKEN(r, t)</code>	shared adapter
4	<code>$S \leftarrow \text{BUILDSLOTMAPPING}(r)$</code>	shared runtime
5	<code>RECLAIMASYNC(r)</code>	shared runtime
6	<code>DECODE(r, S)</code>	worker

Policy specialization. H2O selects lowest-score tokens; Scissorhands retains tokens with persistent attention patterns and evicts low-persistence tokens; Random samples from evictable tokens.

Integration footprint.

Block-native	scheduler/worker	4–6 files, 500+ LOC
vToken	policy adapter	1–2 files, <50 LOC

Figure 6. Policy-adapter procedure and integration footprint in vToken.

5 Evaluation

Our evaluation answers six questions: whether token-level eviction leaves physical capacity trapped in partially live blocks (§5.2), whether vToken improves the paired eviction frontier under identical token-level decisions (§5.3), whether it extends the active-KV capacity frontier under memory pressure (§5.4), where overhead and overlap costs arise (§5.5), how sensitive the system is to runtime parameters (§5.6), and whether it remains compatible with production serving features such as prefix caching while preserving relocation correctness (§5.7–§5.8).

5.1 Experimental Setup

Platform, workloads, and baselines. We evaluate vToken on a single NVIDIA H100 GPU with 80 GB of memory. The main evaluation uses Mistral-7B and Llama-3.1-8B on ShareGPT and LongBench workloads; §5.4 additionally reports a Qwen2.5-14B capacity-frontier check. We consider three token-level KV eviction policies: H2O, Scissorhands, and Random, and distinguish three system variants. NATIVE

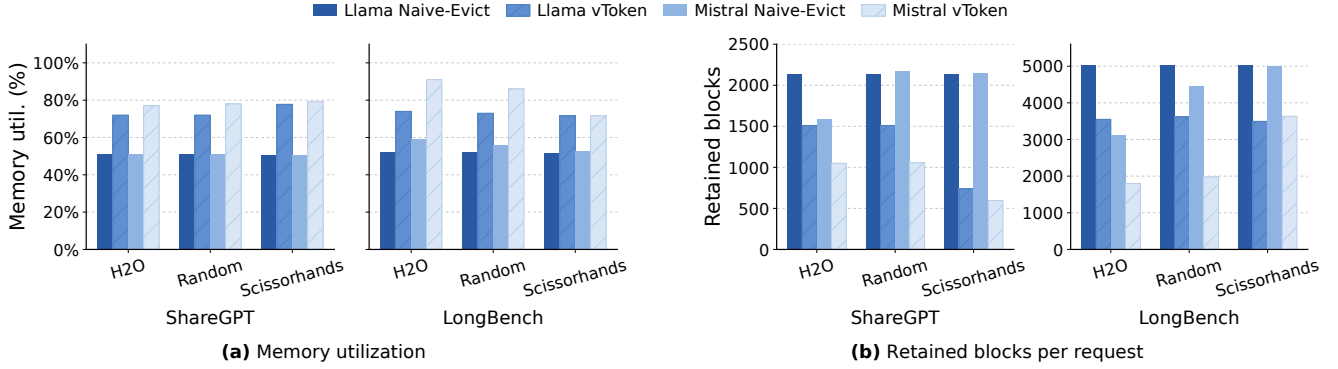


Figure 7. Memory efficiency under token-level eviction. vToken improves effective KV cache usage by reclaiming partially live blocks that remain allocated in NAIVE-EVICT.

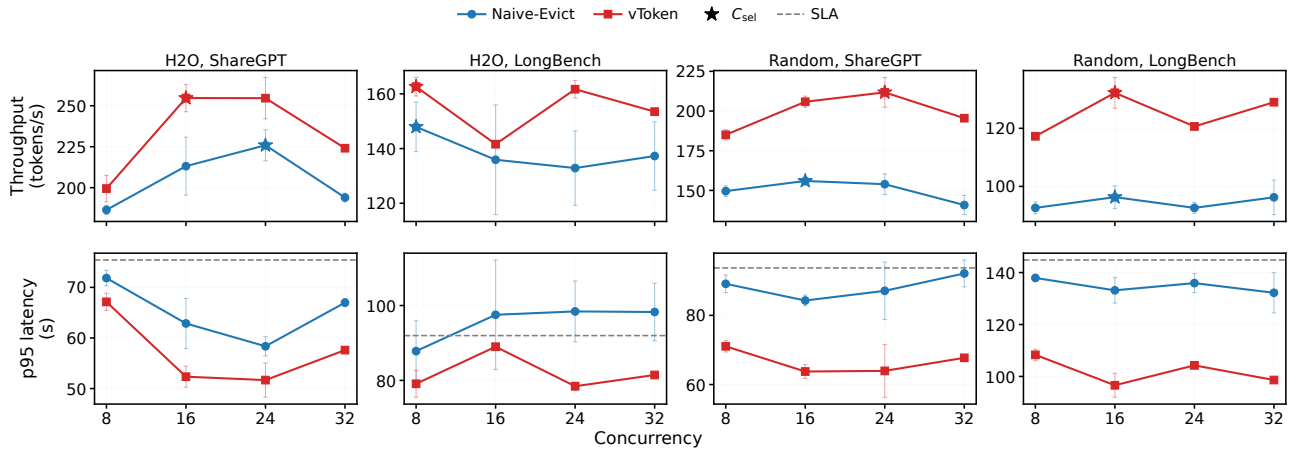


Figure 8. SLA-constrained throughput frontier on Mistral-7B under token-level eviction. The SLA threshold is $1.05 \times$ the p95 latency of NAIVE-EVICT at $C_{ref} = 8$, and the star marks C_{sel} , the maximum-throughput feasible point.

vLLM is the unmodified full-retention serving baseline without vToken hooks or token-level eviction. NAIVE-EVICT applies the same token-level eviction decisions on the block-based runtime but disables vToken’s physical reclamation backend, so partially live blocks remain allocated. vToken enables token-table indirection and physical reclamation on top of the same eviction policies. All reported experiments run with CUDA Graph enabled.

Methodology. For paired comparisons between NAIVE-EVICT and vToken, both variants use the same prompts, decoding settings, model, policy, and memory budget; the only difference is whether token-table indirection and physical reclamation are enabled. We set the decoding temperature to 0 and use `gpu_mem_util=0.90` except in the capacity-frontier experiment, which uses controlled KV budgets as explained in §5.4. Throughput experiments use a closed-loop concurrency scan with the SLA threshold defined from the NAIVE-EVICT baseline at a reference concurrency. The vLLM version used in our prototype requires block sizes of at least

16 tokens, so the block-size sweep in §5.6 is restricted accordingly.

5.2 Memory Efficiency

We compare NAIVE-EVICT and vToken across all model-dataset-policy combinations. This paired comparison isolates physical reclamation: both variants apply the same token-level eviction decisions. We report two complementary metrics: memory utilization measures how effectively allocated KV blocks store retained tokens, while retained blocks per request measure the physical KV footprint that determines admission headroom under memory pressure.

Figure 7a shows that vToken improves memory utilization by translating token-level liveness information into physical block reclamation. Compared with NAIVE-EVICT, vToken increases average memory utilization by 21.88% on Llama-3.1-8B and by 21.67% on Mistral-7B. The gain appears consistently across workloads and policies, indicating that the

effect is not tied to a single model or prompt-length distribution. As shown in Figure 7b, vTOKEN also reduces the number of retained blocks by 27.2%–72.3% relative to NAIVE-EVICT. This block-level reduction is the key systems effect: in NAIVE-EVICT, a block remains allocated as long as it contains any retained token, so token-level holes do not become reusable capacity. vTOKEN compacts the remaining live tokens into fewer physical blocks and returns emptied blocks to the allocator. These results show that the main loss in NAIVE-EVICT is not the eviction policy itself, but the inability of a block-granular runtime to turn token-level liveness into reusable physical capacity. Retained blocks are the more operational metric: they directly determine how many active requests can remain resident before the KV block pool becomes the admission bottleneck. Because decoding is typically memory-bound, this reclaimed KV memory can support more concurrent requests. We evaluate this system-level effect next.

5.3 Sustainable Throughput under Token-Level Eviction

We evaluate whether the block-level capacity reclaimed in §5.2 translates into SLA-constrained throughput improvement. The SLA threshold is $1.05\times$ the NAIVE-EVICT p95 latency at $C_{\text{ref}} = 8$: anchoring to the paired baseline gives both variants the same latency budget, and the 5% slack admits minor tail-latency variance without absorbing regressions. Figure 8 plots Mistral-7B; Llama-3.1-8B results are summarized below. Across all plotted workload-policy combinations, vTOKEN consistently shifts the feasible frontier upward at C_{sel} .

On Mistral-7B, vTOKEN improves selected feasible throughput by 9.9%–37.3% and reduces p95 latency by 9.9%–27.5%. The largest gains occur under Random eviction, where live tokens are scattered across blocks and NAIVE-EVICT retains many partially live blocks; H2O yields smaller but consistently positive gains because its retained tokens are more structured. We observe the same trend on Llama-3.1-8B: across six workload-policy combinations, vTOKEN improves selected feasible throughput by 18.9% on average and reduces p95 latency by 14.7%. The gain is strongest under Random eviction, reaching 37.0% on ShareGPT and 35.9% on LongBench, while H2O still shows gains of 12.5% and 5.7%. Scissorhands shows stronger gains: across Mistral-7B and Llama-3.1-8B, vTOKEN improves selected feasible throughput by 33.3%–103.7% and reduces p95 latency by 21.8%–33.0%. This is expected because Scissorhands retains tokens according to persistent attention patterns, leaving live tokens more dispersed across physical blocks.

Figure 8 marks C_{sel} , the feasible point with the highest throughput under the SLA constraint. The selected point need not be the largest tested concurrency: the gains reported above are realized at C_{sel} , confirming that vTOKEN improves the operating frontier rather than merely pushing

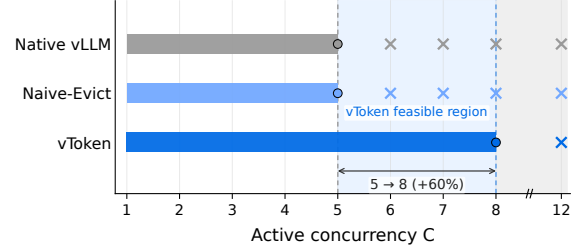


Figure 9. Active-KV capacity frontier at $\text{gpu_mem_util}=0.35$. vTOKEN extends the verified feasible region from $C = 5$ to $C = 8$; $\times$ marks infeasible points.

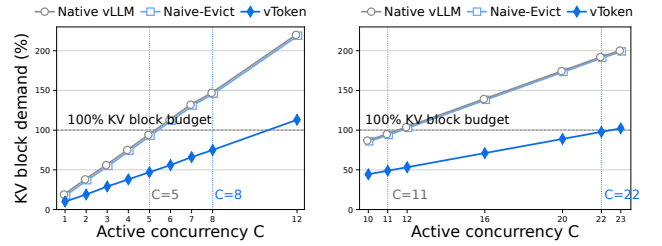


Figure 10. KV-block demand normalized by usable budget. At $\text{gpu_mem_util}=0.35$ (left) and 0.50 (right), vTOKEN extends the verified boundary from $C = 5$ to $C = 8$ and from $C = 11$ to $C = 22$.

to larger batches. Thus, the SLA-frontier gain comes from reclaiming capacity that already exists logically after eviction but remains physically trapped in partially live blocks without vTOKEN.

5.4 Capacity Frontier under Memory Pressure

This experiment asks whether physical reclamation increases the number of active requests that can be simultaneously resident under the same KV block budget. It isolates the active-KV capacity frontier rather than vLLM’s admission queue: a point is infeasible when the target active requests cannot fit in the active KV block pool. We compare NATIVE vLLM, NAIVE-EVICT, and vTOKEN under the same allocator budget; vTOKEN’s evacuation headroom is charged against the same usable block pool rather than allocated as extra memory. For feasible points, throughput is averaged over three runs.

We use Llama-3.1-8B with LongBench, H2O, and `output_len=12288`. The H100 exposes 5,427 usable KV blocks at $\text{gpu_mem_util}=0.35$ and 11,519 at 0.50 . In this setup, a fully retained request needs 1,020 blocks, whereas the ideal packed footprint after H2O retention is 512 blocks. More generally, for active context length L , block size B , and $R(L)$ retained tokens after eviction, full retention scales as $C\lceil L/B \rceil$ blocks, while an ideally packed token-evicted cache scales as

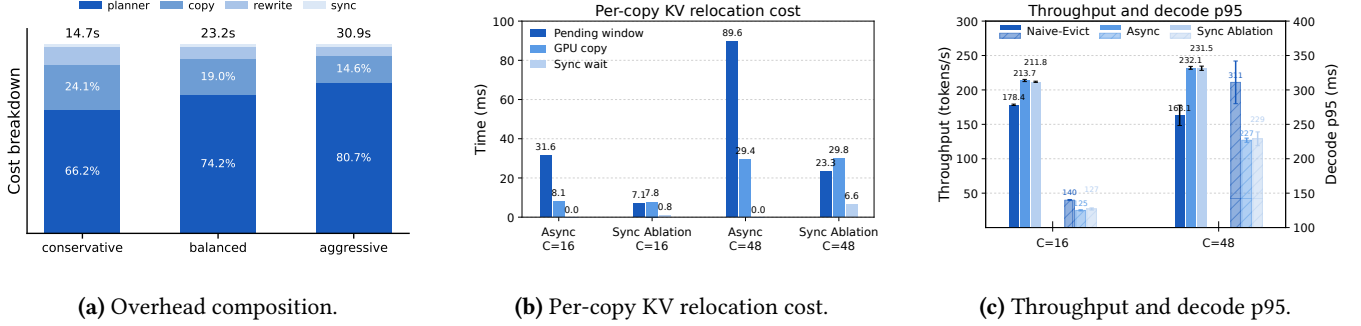


Figure 11. Physical reclamation overhead and overlap. (a) Planner-side work dominates CPU-observed overhead. (b) Async copies avoid explicit synchronization waits. (c) Async reclamation improves throughput and decode p95 over NAIVE-EVICT while staying close to force-sync without explicit waits.

$C[R(L)/B]$. NAIVE-EVICT tends toward the former because partially live blocks remain allocated; vTOKEN moves toward the latter by turning retained tokens into reclaimable physical blocks.

Figure 9 reports the capacity frontier, while Figure 10 explains the underlying block-demand mechanism. At $\text{gpu_mem_util}=0.35$, NATIVE vLLM and NAIVE-EVICT are feasible only through $C = 5$. Six full-retention requests would require 6,120 blocks and exceed the 5,427-block pool, and NAIVE-EVICT remains close to this demand because live tokens stay scattered across partially live blocks. vTOKEN closes the gap toward the 512-block packed footprint, remains feasible through $C = 8$, and extends the maximum feasible concurrency by 60%. Its boundary throughput at $C = 8$ is 180.3 tokens/s, close to its own peak of 203.2 tokens/s at $C = 5$, showing graceful degradation rather than collapse.

The same mechanism persists under the larger controlled budget. At $\text{gpu_mem_util}=0.50$, NATIVE vLLM and NAIVE-EVICT reach their verified frontier at $C = 11$, whereas vTOKEN remains below the usable KV-block budget through $C = 22$, doubling the verified feasible concurrency. Figure 10 shows that the shift comes from lower per-request physical block demand under the same allocator budget, not from extra headroom or a different admission policy.

Larger-model capacity check. We repeat the capacity-frontier sweep on Qwen2.5-14B with $\text{output_len}=8192$ and $\text{gpu_mem_util}=0.50$, leaving 3,261 usable KV blocks. NATIVE vLLM and NAIVE-EVICT are feasible only through $C = 3$ and first fail at $C = 4$, whereas vTOKEN remains feasible through $C = 6$ and fails at $C = 8$. This $2\times$ frontier extension indicates that the capacity benefit is not specific to Llama-3.1-8B. This check isolates capacity-frontier extension: native full-KV serving remains preferable when feasible, while vTOKEN extends the feasible region once full retention exhausts the KV block budget. The sensitivity of this boundary to the eviction ratio is examined in §5.6.

5.5 Overhead and Overlap

This subsection separates CPU-observed runtime overhead from GPU data-movement behavior and checks whether the physical reclamation backend overlaps with decoding. We run this mechanism experiment on Llama-3.1-8B with LongBench and H2O. We first verify that the steady-state indirection path is nearly free: an indirection-only ablation that installs the vTOKEN hooks but disables eviction and reclamation, with slot translation refreshed only on structural changes, changes throughput and p95 by less than 1.0% relative to NATIVE vLLM at both $C=16$ and $C=48$. The overhead reported below therefore comes from the pressure-activated planning and relocation backend, not from per-attention table lookup. Figure 11(a) reports planner work, host-side copy launch/accounting, and token/block-table rewrites across 100 profiling runs. Planning consistently dominates the CPU-observed overhead, indicating that the main measured cost is planner-side opportunity checking rather than KV relocation accounting.

Figure 11(b) validates the asynchronous path with a force-synchronization ablation. In the normal path, relocation copies incur no explicit synchronization wait: the copy remains pending for multiple decode steps, while GPU copy time is much shorter than the pending window. In contrast, the force-sync ablation exposes CPU blocking per relocation event, confirming that async overlap avoids an explicit synchronization stall.

The overlap is still not free, since copy kernels share GPU resources with decode. Figure 11(c) shows that async reclamation improves both throughput and decode p95 over NAIVE-EVICT at $C = 16$ and $C = 48$, while staying close to force-sync without exposing synchronization stalls. Since reclamation is less frequent at the capacity-frontier operating points, these measurements should be read as a stress test of the overlap mechanism rather than the steady-state cost at the frontier. Thus, vTOKEN removes explicit stalls and

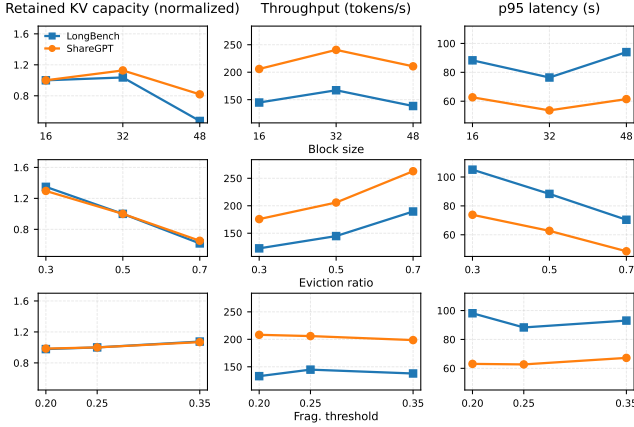


Figure 12. Sensitivity analysis of block size, eviction ratio, and fragmentation threshold under H2O.

overlaps copy with decode, but copy/decode contention can appear under heavier operating pressure.

5.6 Sensitivity Analysis

We use one-factor-at-a-time experiments to evaluate how vToken responds to three runtime parameters: block size, eviction ratio, and fragmentation threshold. Figure 12 reports normalized retained KV capacity, throughput, and p95 latency under fixed concurrency for H2O on LongBench and ShareGPT. The retained KV capacity is computed as retained blocks multiplied by block size, and is normalized to the default configuration for each model–dataset pair. We show H2O as the representative policy because StreamingLLM and Random exhibit the same qualitative trends across these sweeps.

Block size. We start from block size 16 because it is the minimum supported by vLLM’s PagedAttention allocator in our evaluated configuration. Changing the block size affects both allocation granularity and the physical capacity represented by each block. Normalized retained KV capacity therefore provides a comparable view across block sizes. Both workloads achieve the best throughput and lowest p95 latency at block size 32, while block size 48 reduces retained capacity but degrades performance. This shows that simply enlarging blocks creates a capacity/performance tradeoff and is not a substitute for token-level reclamation.

Eviction ratio. Increasing the eviction ratio exposes more reclaimable KV capacity. As the ratio increases from 0.3 to 0.7, normalized retained KV capacity decreases substantially on both workloads, while throughput improves and p95 latency drops consistently. This shows that vToken can convert more aggressive token eviction into physical KV-cache reclamation. This sweep stresses the reclamation mechanism; selecting an accuracy-preserving eviction budget remains the responsibility of the underlying eviction policy.

Table 1. Prefix-cache compatibility.

Degree	Shared skipped	Suffix reclaimed	Block red.	Throughput Δ	p95 Δ
1 (no sharing)	n/a	535K	28.6%	-2.9%	+2.9%
2	100%	65.6K	33.0%	+76.1%	-40.2%
4	100%	198K	32.3%	+97.2%	-40.9%
8	100%	264K	42.2%	+146.5%	-47.8%

Fragmentation threshold. The fragmentation threshold is less dominant than the eviction ratio. Raising the threshold delays reclamation slightly, which modestly increases normalized retained KV capacity and mildly affects p95 latency, while throughput remains comparatively stable across the evaluated range. This indicates that vToken does not require fine-grained trigger tuning: the main effect comes from exposing token-level holes, while the reclamation trigger controls when physical reclamation is worthwhile.

5.7 Prefix-Cache Compatibility

Prefix caching is a common production serving feature. vToken’s current implementation conservatively excludes shared-prefix blocks from relocation while keeping private suffix blocks eligible for reclamation. We validate this boundary on Llama-3.1-8B using synthetic H2O workloads with an 8K-token shared prefix, an 8K-token private suffix, and sharing degrees from 1 to 8.

Table 1 shows that for sharing degrees 2, 4, and 8, all shared-prefix candidates are skipped and all consistency checks pass. Relative to Naive-Evict with prefix caching enabled, vToken reduces retained blocks by 28.6%–42.2%; because both variants use prefix caching, the gains come from private-suffix reclamation rather than different hit rates. Higher sharing degrees amplify this effect because protected shared-prefix blocks leave private suffixes as the main reclaimable capacity under pressure. The degree-1 case has no shared-prefix reuse and shows only the small overhead of the conservative path. A less conservative extension could use copy-on-write for shared blocks when the expected reclamation benefit justifies the extra copy.

5.8 Relocation Correctness and Generation Stability

This experiment is the empirical test of invariants **I1** (token conservation) and **I2** (unique mapping) from §3.5. For each relocation event, we hash the ordered set of retained token IDs together with their K/V tensor contents and compare the hash before and after relocation; we additionally check that no logical token is lost or duplicated and that every (block, offset) referenced by the token table is valid. Across all evaluated workloads and policies, these checks hold for every relocation event, confirming that vToken’s indirection and asynchronous copies preserve the retained logical KV state of every request. The check is deliberately scoped to the runtime mechanism: because Naive-Evict and vToken apply identical eviction decisions, any task-quality change

Table 2. Generation stability under physical reclamation.

Metric	Result	Stability
ROUGE-L F1	mean $\Delta = -0.0016$	$93.1\% \leq 0.01$
Output length	median $v\text{TOKEN}/\text{NAIVE-EVICT} = 0.99$	no inflation

attributable to dropping tokens is a property of the eviction policy, not of $v\text{TOKEN}$.

Beyond exact KV-state preservation, we also check whether $v\text{TOKEN}$ ’s token-table indirection and physical reclamation introduce observable generation-quality degradation. We run Llama-3.1-8B on ShareGPT under deterministic decoding, and compare $v\text{TOKEN}$ against NAIVE-EVICT using 144 matched generations. As shown in Table 2, the mean paired ROUGE-L F1 difference is only -0.0016 , and 93.1% of the pairs differ by at most 0.01. The median $v\text{TOKEN}/\text{NAIVE-EVICT}$ output-length ratio is 0.99, indicating no systematic length inflation. This paired comparison suggests that $v\text{TOKEN}$ ’s indirection does not add measurable quality drift beyond the eviction policy itself.

6 Discussion

$v\text{TOKEN}$ is a runtime-boundary abstraction rather than a $v\text{LLM}$ -specific optimization. It targets PagedAttention-style runtimes where KV memory is managed in blocks and attention kernels access KV entries through mutable slot mappings. Porting $v\text{TOKEN}$ requires hooks for block allocation and release, pre-attention slot-map updates, and asynchronous KV copy with dependency tracking; our implementation instantiates these hooks in $v\text{LLM}$.

Deployment scope. In production, NATIVE $v\text{LLM}$ remains the preferred low-pressure path, while $v\text{TOKEN}$ is a pressure-activated extension for regimes where token eviction must translate into reusable physical capacity. The capacity-frontier result measures net usable capacity under admission-controlled headroom, where bounded relocation workspace is reserved within the same KV block budget.

Scope and extensibility. Our current implementation uses one runtime KV cache group and conservatively skips shared-prefix blocks to preserve prefix-cache correctness. These choices define the current implementation scope; the abstraction boundary remains the same. For MHA [32], GQA [2], and MQA [29] models, $v\text{TOKEN}$ ’s logical token view is unchanged: the number of KV heads changes the physical tensor shape, but not token identity or slot-remapping semantics. Runtimes with multiple KV cache groups can share logical liveness state while keeping group-specific placement arrays. Shared-prefix support keeps private suffix blocks eligible by default and could use copy-on-write for shared blocks only when the expected reclamation benefit justifies the extra copy. In tensor-parallel settings, each TP shard has an independent KV block pool, so token-table indirection, slot remapping, and reclamation would operate on shard-local block IDs; cross-shard coordination can remain in the

existing scheduler. Cross-device strategies that explicitly account for transfer cost remain orthogonal to the granularity mismatch we address.

Backend constraints and overhead. Policy inputs depend on backend-exposed signals such as attention scores; the $v\text{TOKEN}$ substrate itself is independent of attention implementation. Our overhead results show that planner-side opportunity checks dominate measured overhead, while the overlap study shows that asynchronous copies avoid explicit synchronization stalls but can still contend with decode. Batching and indexing these checks within the runtime is an engineering optimization target.

7 Related Work

We review prior work in three areas that define the design space around $v\text{TOKEN}$: block-based LLM inference systems, KV cache optimization mechanisms, and memory-virtualization abstractions.

LLM Inference Systems. $v\text{LLM}$ [17] introduced Page-dAttention, a block-based KV cache manager that reduces external fragmentation and enables prefix sharing. This design has been widely adopted in systems such as TENSORRT-LLM [25] and LIGHTLLM [24]. These systems provide an efficient physical substrate, but their memory-management interface remains block-oriented: allocation, reclamation, and slot mapping are expressed in terms of blocks rather than individual tokens. Other systems improve serving through parallelism, scheduling, or offloading [3, 30], but do not provide a runtime abstraction that lets token-level eviction policies reclaim partially used KV blocks.

KV Cache Optimization. KV cache optimization techniques reduce memory pressure by changing cache contents, cache representation, or the attention computation that consumes the cache. Quantization and compression approaches [6, 11, 14, 15, 20, 23] reduce KV representation size and are orthogonal to $v\text{TOKEN}$. Token eviction policies such as H2O [36], STREAMINGLLM [33], SCISSORHANDS [22], and FASTGEN [11] decide *which* tokens should be retained; their main contribution is policy design, not the memory substrate needed to realize those decisions on a block-based serving runtime.

Several recent systems move closer to runtime KV management. CACHEGEN [21] targets compact cache transfer and QUEST [31] reduces attention cost by selecting useful KV entries. DIFFKV [35] differentiates K/V precision, token retention, and per-head layout to compress the cache itself, requiring a mixed-precision page manager; $v\text{TOKEN}$ ’s substrate operates on uniform fp16 KV and is orthogonal to such representation changes. PAGEDEVICT [9] shifts the alignment burden to the policy by making eviction page-aligned, which constrains policy-side decisions and does not directly accommodate existing token-level policies such as H2O/StreamingLLM whose importance ordering is unrelated

to page boundaries; vTOKEN keeps policies page-agnostic and resolves alignment in the runtime. A recent concurrent system, ZIPAGE [18], enforces a fixed per-request KV-block budget by relocating retained entries into bounded blocks. It is a concrete bounded-cache pipeline for reasoning workloads, whereas vTOKEN exposes a policy-neutral virtualization boundary that keeps eviction policies page-agnostic and delegates slot remapping and physical reclamation to the runtime. Overall, these systems either change the cache representation, constrain eviction granularity, or build a bounded-cache pipeline; vTOKEN instead preserves the block substrate and inserts the missing token-level virtualization boundary above it.

Memory Virtualization. In LLM serving, vATTENTION [28] virtualizes the *address space* at page granularity using CUDA VMM APIs, which avoids specialized paged-attention kernels but cannot reclaim intra-page holes left by token-level eviction. vTOKEN virtualizes *token liveness* at token granularity above the block substrate, exposing per-token retention to the runtime so that intra-block holes can be physically reclaimed. The two layers are stackable: vATTENTION can serve as the address-space backend while vTOKEN handles token-level reclamation on top. vTOKEN specifically targets the missing layer between token-level eviction policies and block-based KV cache managers.

8 Conclusion

We introduced vTOKEN, a token-level virtualization layer that makes token-level KV eviction physically effective in block-managed LLM serving runtimes. By decoupling logical token liveness from physical placement, vTOKEN lets policies express which tokens to remove while the runtime handles slot remapping and physical reclamation. Our vLLM-based instantiation of vTOKEN reduces retained KV blocks by 27.2%–72.3% and improves SLA-constrained throughput by up to 1.37 $\times$ over NAIVE-EVICT; under constrained active-KV capacity, it extends the maximum feasible concurrency by up to 2 $\times$. These results show that a stable logical KV view can make token-level eviction practical in existing block-based serving systems.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv:2305.13245 [cs.CL]* <https://arxiv.org/abs/2305.13245>
- [3] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. 2022. DeepSpeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [4] Anthropic. 2026. Claude Code. <https://github.com/anthropics/claude-code>. GitHub repository, accessed April 22, 2026.
- [5] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *arXiv:2308.14508 [cs.CL]* <https://arxiv.org/abs/2308.14508>
- [6] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, et al. 2024. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069* (2024).
- [7] Wei-Lin Chiang, Zhuohan Li, Ziqing Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E Gonzalez, et al. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality. See <https://vicuna.lmsys.org> (accessed 14 April 2023) 2, 3 (2023), 6.
- [8] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E Gonzalez, et al. 2024. Chatbot arena: An open platform for evaluating llms by human preference. *arXiv preprint arXiv:2403.04132* (2024).
- [9] Krishna Teja Chitty-Venkata, Jie Ye, Siddhisanket Raskar, Anthony Kougkas, Xian Sun, Murali Emani, Venkatram Vishwanath, and Bogdan Nicolae. 2026. PagedEviction: Structured Block-wise KV Cache Pruning for Efficient Large Language Model Inference. In *Findings of the Association for Computational Linguistics: EACL 2026*, Vera Demberg, Kentaro Inui, and Lluís Marquez (Eds.). Association for Computational Linguistics, Rabat, Morocco, 3207–3218. doi:10.18653/v1/2026.findings-eacl.168
- [10] Dom Eccleston. 2023. ShareGPT: Share your ChatGPT conversations with one click. <https://github.com/domeccleston/sharegpt>. GitHub repository.
- [11] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2024. Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs. In *The Twelfth International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2310.01801>
- [12] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shihong Ma, Xiao Bi, et al. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645, 8081 (2025), 633–638.
- [13] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yifan Wu, YK Li, et al. 2024. DeepSeek-Coder: when the large language model meets programming—the rise of code intelligence. *arXiv preprint arXiv:2401.14196* (2024).
- [14] Yefei He, Luoming Zhang, Weijia Wu, Jing Liu, Hong Zhou, and Bohan Zhuang. 2024. Zipcache: Accurate and efficient kv cache quantization with salient token identification. *Advances in Neural Information Processing Systems* 37 (2024), 68287–68307.
- [15] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun S Shao, Kurt Keutzer, and Amir Gholami. 2024. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems* 37 (2024), 1270–1303.
- [16] Tom Kilburn, David BG Edwards, Michael J Lanigan, and Frank H Sumner. 2009. One-level storage system. *IRE Transactions on Electronic Computers* 2 (2009), 223–235.
- [17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [18] Mengqi Liao, Lu Wang, Chaoyun Zhang, Bo Qiao, Si Qin, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Huaiyu Wan. 2026. Zipage: Maintain High Request Concurrency for LLM Reasoning through

- Compressed PagedAttention. arXiv:2603.08743 [cs.DC] <https://arxiv.org/abs/2603.08743>
- [19] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [20] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. 2024. Minicache: Kv cache compression in depth dimension for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 139997–140031.
- [21] Yuhao Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.
- [22] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems* 36 (2023), 52342–52364.
- [23] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750* (2024).
- [24] ModelTC. 2026. LightLLM. <https://github.com/ModelTC/lightllm>. GitHub repository, accessed April 22, 2026.
- [25] NVIDIA. 2026. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM>. GitHub repository, accessed April 22, 2026.
- [26] OpenAI. 2026. Codex: Lightweight Coding Agent. <https://github.com/openai/codex>. GitHub repository, accessed April 22, 2026.
- [27] OpenClaw Contributors. 2026. OpenClaw: Personal AI Assistant Gateway. <https://github.com/openclaw/openclaw>. GitHub repository, accessed April 22, 2026.
- [28] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vattention: Dynamic memory management for serving llms without pagedattention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 1133–1150.
- [29] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150 [cs.NE] <https://arxiv.org/abs/1911.02150>
- [30] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*. PMLR, 31094–31116.
- [31] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. Quest: Query-aware sparsity for efficient long-context llm inference. *arXiv preprint arXiv:2406.10774* (2024).
- [32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. arXiv:1706.03762 [cs.CL] <https://arxiv.org/abs/1706.03762>
- [33] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453* (2023).
- [34] Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Mailthody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. 2025. Strata: Hierarchical Context Caching for Long Context Language Model Serving. arXiv:2508.18572 [cs.DC] <https://arxiv.org/abs/2508.18572>
- [35] Yanqi Zhang, Yuwei Hu, Runyuan Zhao, John C. S. Lui, and Haibo Chen. 2025. DiffKV: Differentiated Memory Management for Large Language Models with Parallel KV Compaction. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (Lotte Hotel World, Seoul, Republic of Korea) (SOSP '25)*. Association for Computing Machinery, New York, NY, USA, 431–445. doi:10.1145/3731569.3764810
- [36] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems* 36 (2023), 34661–34710.
- [37] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.